EE 250L: Distributed Systems for the Internet of Things

Fall 2026 - Lab 5

Wireless Signal Measurements

	Tuesday Lab	Thursday Lab
Assigned Date	9/29/26	10/1/26
Submission Due Date	10/5/26 (3 PM)	10/7/26 (3 PM)
Interview + Demo Due Date	10/6/26 (Before Lab Starts)	10/8/26 (Before Lab Starts)

Lab outcome:

1. Measure Wi-Fi signal strength (RSSI) at several locations with a Python script on your PC and plot mean and standard deviation.
2. Run an iPerf 2 server on the Ubuntu VM and use the ESP32-S3 as a moving iPerf client to measure TCP and UDP throughput and packet loss at five distances.
3. Log the results to CSV and plot TCP vs UDP throughput per distance.

Files to submit: `main.py`, `signal_strength.png`, `iperf_plot.py`, the five throughput plots, the `iperf_tcp_<d>m.csv` / `iperf_udp_<d>m.csv` files, and `README.md` (answers to the 14 questions and acknowledgment of assistance). See section 8.

Introduction

In this lab, you'll measure the WiFi signal strength at different locations in your house (or whatever building you're working in) to see how signal strength changes as you move away from the router, around barriers, etc.

Code

Go to the lab5-demo in Google Drive and download to your PC. The ESP32-S3 firmware for Part 2 is in the `esp32_iperf_client` folder next to it. You need your PC with Python 3 (Part 1), your Ubuntu VM in bridged networking mode as the iPerf server (Part 2), and your ESP32-S3 board with the ESP-IDF setup from Lab 3.

Part One - Wireless Signal Measurements

You will need to do this lab on your PC (not your VM or ESP32-S3) UNLESS OTHERWISE MENTIONED. First, you'll need some python packages that don't come with the standard library. These requirements are listed in the requirements.txt file in the repository (in lab5-demo). To install them, you can run

```
pip install -r requirements.txt
# macOS / many Linux systems: the commands are pip3 and python3
pip3 install -r requirements.txt
```

(On macOS the commands are `pip3` and `python3`; the script runs `sudo wdutil info` to read the RSSI, so it will ask for your password.)

Open main.py and read through the comments, documentation, and code. Answer all the questions in the comments (put the answers in your README.md file, as usual) and implement the TODOs.

Experiment Setup

1. Identify at least five different locations where you will measure WiFi signal strength.
2. At each location, collect ten samples of the signal strength.
3. Compute the mean and standard deviation of the collected signal strengths.
4. Visualize the results in a bar chart with error bars.

Function: Get WiFi Signal Strength

- This function retrieves the WiFi signal strength in dBm.
- Uses OS-specific commands to extract signal strength.
- On Windows, the percentage quality is converted to dBm using `-100 + quality/2`.

Function: Main Execution

- Prompts the user to go to different locations and press enter to start sampling.
- Records 10 samples of signal strength at each location.
- Computes mean and standard deviation.
- Uses Plotly to create a bar chart with error bars.

Out of the 5 reading at least 3 of them need to have significant variation in comparison to one another, Do Not Slack and take measurements casually.

There are a total of 9 questions in the main.py file which you'd need to include in your Readme files.

Part 2: Measuring Network Performance Using iPerf

Measuring TCP and UDP Performance with iPerf

(Main shell script to generate csv for TCP UDP throughput tests)

Objective

In this lab, students will:

✅ Set up network performance measurement using iPerf.

✅ Compare TCP vs. UDP throughput and packet loss.

Throughput here refers to the Actual speed of successful data transfer.

✅ Understand how distance affects Wi-Fi performance.

✅ Analyze and visualize logged data.

Wi-Fi networks adapt their transmission rates based on signal quality. As devices move further from the router, the signal strength decreases, affecting throughput, reliability, and packet loss.

What We Are Testing (Here we will use Laptop VM & ESP32-S3)

1. We keep the Laptop VM stationary near the router.
2. We move the ESP32-S3 to 5 different distances from the router (e.g., 2m, 4m, 6m, 7m, 8m) while sending dummy network packets to the server on VM.

What is Being Sent from ESP32-S3 (Sender) to VM (Receiver)? i.e When running iPerf, the ESP32-S3 (client/sender) sends dummy network traffic (not actual files or meaningful data) to the VM (server/receiver). The goal is to measure network performance, not transfer real content.

3. We measure how throughput changes in TCP and UDP tests.

Expected Observations

Distance	Expected TCP Behavior	Expected UDP Behavior
2m, 5m (Near Router)	High throughput, minimal packet loss	High throughput, near-zero packet loss
10m (Moderate Distance)	Reduced throughput due to lower signal strength	Some packet loss starts appearing
15m+ (Weak Signal)	Unstable throughput	Increased latency, Significant packet loss, potential disconnections

Key Learning Points

- TCP adapts to weak signals by slowing down and retransmitting lost packets.
- UDP does not re-transmit, so packet loss increases significantly at larger distances.
- Wi-Fi degrades with range, and you will observe this using real experiments.

1. Required Equipment

- VM (Ubuntu/Linux) → Acts as the iPerf server.
- ESP32-S3-DevKitC-1 (ESP-IDF v6.0.2 as installed in Lab 3; v5.3 also builds this project) → Acts as the iPerf client.
- Wi-Fi Router → Connect both devices to the same network (2.4 GHz, the ESP32-S3 has no 5 GHz radio).
- Power Banks for wirelessly powering the ESP32-S3. (its cheap and also please share if needed)

1.1 Network Configuration

1. Connect both the ESP32-S3 and Laptop VM to the same Wi-Fi network (VM in bridged networking mode, as in Lab 2 Part 3).
2. Find their IP addresses using:

```
hostname -I                          # on the VM
# the ESP32-S3 prints its IP in the serial monitor: wifi: got IP
192.168.1.50 ...
```

Ensure VM and ESP32-S3 can see each other using ping:

```
From VM: ping -c 5 <ESP32_ip>
```

Expected output: `64 bytes from 192.168.1.100: time=2.5 ms`

2. Install iPerf

Run the following command on the VM. Note: the ESP32-S3 speaks the iPerf **2** protocol, so install `iperf` (version 2), not `iperf3`:

```
sudo apt update
sudo apt install iperf -y
iperf -v                             # must say "iperf version 2.x"
```

On the ESP32-S3, the iPerf client is the firmware in `esp32_iperf_client` (supplied complete, no TODOs). Fill in `main/credentials.h` (Wi-Fi SSID, password, and the VM's IP address), then build and flash as in the previous labs:

```
idf.py set-target esp32s3 && idf.py build && idf.py -p <PORT> flash monitor
```

Replace `<PORT>` with your board's serial port: `ls /dev/cu.*` (macOS, e.g. `/dev/cu.usbmodem1101`), `ls /dev/ttyUSB* /dev/ttyACM*` (Linux) or Device Manager → Ports (Windows, e.g. `COM5`). The name can change when you re-plug the board, so check it again if flashing says the port does not exist. If flashing says `No serial data received`, hold BOOT, tap RESET, release BOOT and run the command again.

If you use a phone hotspot as the access point: on an iPhone turn **Maximize Compatibility** on (the board is 2.4 GHz / WPA2 only), turn Low Power Mode off, and keep the Personal Hotspot screen open while the board connects. A hotspot that has gone idle drops the WPA2 handshake, which the monitor shows as `disconnected (reason=204)` even with the right password; toggling the hotspot off and on fixes it.

The serial monitor runs on your PC in the ESP-IDF terminal (as in Labs 3 and 4). After the board connects to Wi-Fi it shows a banner with the board's IP and then an `esp32>` prompt (press RESET once if you only see the ROM boot lines). Type `rssi` to check that it responds. **All ESP32-S3 commands in sections 4 and 5 are typed at this prompt**; all server commands are typed in the VM terminal. If you closed the monitor, reopen it (from the `esp32_iperf_client` folder, with the ESP-IDF environment loaded) and press RESET:

```
idf.py -p <PORT> monitor           # Ctrl+] closes it
```

Keep the default sdkconfig; do not change the console channel in menuconfig.

3. Start the iPerf Server (On Laptop VM)

On the VM, run:

```
iperf -s
```

This starts a TCP server, listening on port 5001. iPerf 2 serves one protocol per process, so for the UDP test in 4.2 stop the TCP server (`Ctrl+C`) and start the UDP server, or run it in a second terminal:

```
iperf -s -u
```

Stop any server you started by hand (Ctrl+C) before section 5 - the script there starts its own servers and needs port 5001 free.

To disable Firewalls from blocking TCP and UDP traffic, do:

```
sudo ufw disable
```

4. Running TCP & UDP Tests from ESP32-S3

4.1 Running a TCP Test (One-Time)

On the ESP32-S3 (at the `esp32>` prompt), run:

```
iperf -c <VM IP(iperf server)> -t 10
```

Expected output:

```
[TCP] 0.0-10.0 sec  22.61 MBytes  18.09 Mbits/sec  sender
# and on the VM:
[  1] 0.0000-10.0143 sec  22.5 MBytes  18.0 Mbits/sec
```

Key Observations:

- Sender throughput = How fast the ESP32-S3 sent data.
- Receiver throughput = How much data the VM received
- The board's LED is blue during a TCP test, green during a UDP test, red if a test failed (server not running), off when idle.

4.2 Running a UDP Test (One-Time)

On the ESP32-S3, run:

```
iperf -c <VM IP(iperf server)> -u -b 10M -t 10

[UDP] 0.0-10.0 sec  12.50 MBytes  10.00 Mbits/sec  sender  (8504 datagrams)
[UDP] server report: 9.96 Mbits/sec  receiver   lost 31/8504 datagrams (0.36%)
# and on the VM:
[  1] 0.00-10.00 sec  11.9 MBytes  10.0 Mbits/sec   1.727 ms 31/8504 (0.36%)
```

The second line is the **server report**: only the receiver can count lost UDP datagrams, so at the end of the test the board asks the server for its counts and prints them.

5. Automating TCP & UDP Tests with a Script

(Script in Drive, run on the VM with 5 distances)

Now, we will use a Bash script to automate:

✅ Run your TCP tests 1 time at each distance which has 5 test runs, computing the average, and logs the data.

✅ Run your UDP tests 1 time at each distance which has 5 test runs, computing the average, and logs the data.

✅ Print results to the terminal for verification.

5.1 Create & Edit iperf_logger.sh

On the VM, create the script:

```
nano iperf_logger.sh
```

Copy the code from the file in the drive or SCP to transfer the file from local to the VM

(SCP covered before)

This script is already given in your drive folder such that you don't have to write the shell script from scratch. Just run it on the VM as is. (The ESP32-S3 cannot write CSV files, so unlike last year the script runs on the server side: it starts the TCP and UDP servers and logs what the VM receives.) To run the bash file,

```
chmod +x iperf_logger.sh
(this command adds the execute (x) permission to a bash file)
./iperf_logger.sh <distance_Args>
```

5.2 Running the script at each distance

You need two windows: the VM terminal (server side) and the ESP32-S3 serial monitor from section 2 (client side). For each distance:

1. Make sure no `iperf -s` from section 3 is still running (Ctrl+C it).
2. Place the ESP32-S3 at the distance you want to measure (e.g. 2 m from the router). Leave the VM where it is, near the router.
3. In the VM terminal, in the `lab5-demo` folder, start the logger with the distance in metres as the argument, and wait until it prints "Waiting for 5 TCP runs, then 5 UDP runs...":

```
./iperf_logger.sh 2
```

4. In the serial monitor, at the `esp32>` prompt, type

```
measure
```

and press Enter (or, if the board is on a power bank with no laptop attached, press the board's **BOOT** button - it does the same thing). The board runs 5 TCP tests (LED blue) and then 5 UDP tests (LED green), 10 s each, about 2 minutes in total; the LED turns off when it is done.

5. Watch the VM terminal: the script prints one line per run as it arrives ("TCP Receiver Throughput (Run 1): ...", then the UDP runs with their packet loss) and finishes with the averages and "Logging completed. Data saved in iperf_tcp_2m.csv and iperf_udp_2m.csv". The board prints its own sender-side summary at the same time.
6. Move the board to the next distance and repeat from step 3 with the new number, e.g.

```
./iperf_logger.sh 4
./iperf_logger.sh 6
```

And so on as you move ahead. Both TCP and UDP throughput tests are done in the same script, the only change is the distances, logs are stored in the CSVs with respective names of protocols as well as distances for plotting. Always start the script **before** typing `measure`: UDP datagrams sent while no server is listening are silently lost and that run would be logged as 100 % loss (or not at all). If a run goes wrong (server not running, board disconnected), just run the same distance again — each run appends a new row to the CSV and you plot the last row.

On the iPerf server-side implementation (Laptop VM fixed near router)

```
$ ./iperf_logger.sh 1
==========================================
 Logging iPerf tests at 1 meters
 iperf2 TCP + UDP servers listening on 172.20.10.11:5001
 Now type   measure   in the ESP32-S3 serial console.
 Waiting for 5 TCP runs, then 5 UDP runs...
==========================================
TCP Receiver Throughput (Run 1): 3.640 Mbps
TCP Receiver Throughput (Run 2): 3.120 Mbps
TCP Receiver Throughput (Run 3): 3.461 Mbps
TCP Receiver Throughput (Run 4): 3.574 Mbps
TCP Receiver Throughput (Run 5): 3.634 Mbps
UDP Receiver Throughput (Run 1): 9.993 Mbps   Packet Loss: 0.035 %
UDP Receiver Throughput (Run 2): 10.048 Mbps   Packet Loss: 0.223 %
UDP Receiver Throughput (Run 3): 10.041 Mbps   Packet Loss: 0.047 %
UDP Receiver Throughput (Run 4): 2.149 Mbps   Packet Loss: 0.135 %
UDP Receiver Throughput (Run 5): 10.027 Mbps   Packet Loss: 0.000 %
------------------------------------------
TCP Results (Mbps): 3.640 3.120 3.461 3.574 3.634   Average: 3.486 Mbps
UDP Results (Mbps): 9.993 10.048 10.041 2.149 10.027   Average: 8.452 Mbps
Packet Loss (%):    0.035 0.223 0.047 0.135 0.000   Average: 0.088 %
Logging completed. Data saved in iperf_tcp_1m.csv and iperf_udp_1m.csv.
------------------------------------------
```

On the ESP32-S3 Client side (varying distances)

```
esp32> measure
==========================================
 measure: 5 TCP runs then 5 UDP runs, 10 s each, to 172.20.10.11:5001
 RSSI at this spot: -40 dBm
 (the VM must be running ./iperf_logger.sh <distance>)
==========================================
```

```
Starting TCP Tests...
I (380906) led: LED -> blue
TCP Run 1...
TCP Sender Throughput (Run 1): 3.080 Mbps
TCP Run 2...
TCP Sender Throughput (Run 2): 3.588 Mbps
   ...
Starting UDP Tests...
I (436234) led: LED -> green
UDP Run 1...
UDP Sender Throughput (Run 1): 10.000 Mbps   Packet Loss: 0.047 %
   ...
I (491443) led: LED -> off
-------------------------------------------
Sender-side summary (the VM's CSV files hold the receiver side):
TCP: Run1,Run2,Run3,Run4,Run5,Avg = 3.080,3.588,3.508,2.688,2.759,3.125
UDP: Run1,Run2,Run3,Run4,Run5,Avg = 10.000,10.000,10.000,10.000,10.000,10.000
UDP loss %: 0.047,0.059,0.059,0.059,0.071,Avg 0.059
measure completed.
-------------------------------------------
```

6. Logging and Analyzing Results

After successfully running the iPerf tests at different distances, the VM will have generated CSV log files, in the folder where you ran iperf_logger.sh, containing the throughput measurements.

6.1 Locating the Logged Data

The test results will be stored in the following files:

- **TCP Results:** `iperf_tcp_<distance>m.csv`
  - Example: `iperf_tcp_2m.csv`, `iperf_tcp_5m.csv`
- **UDP Results:** `iperf_udp_<distance>m.csv`
  - Example: `iperf_udp_2m.csv`, `iperf_udp_5m.csv`

6.2 Viewing Logged Data

Checking the Latest Entries

To view the last few lines of the logged CSV files:

- For TCP results (replace 2m with the correct distance):

```
tail -n 5 iperf_tcp_2m.csv
```

- For UDP results:

```
tail -n 5 iperf_udp_2m.csv
```

Displaying the Full CSV

To see the entire contents of the log file:

```
cat iperf_tcp_2m.csv
cat iperf_udp_2m.csv
```

6.3 Understanding the CSV File Format

The CSV logs store multiple test runs for each distance:

`iperf_tcp_2m.csv` (Example Data)

```
Distance, Run1, Run2, Run3, Run4, Run5, Avg
2, 92.3, 90.5, 89.1, 91.2, 93.0, 91.2
5, 85.2, 84.0, 83.5, 82.9, 85.1, 84.1
```

Column 1: Distance from the router (in meters).

Run1 – Run5: Throughput (Mbps) from five test runs.

Avg: Average TCP throughput.

`iperf_udp_2m.csv` (Example Data)

```
Distance, Run1, Run2, Run3, Run4, Run5, Avg, PacketLoss1, PacketLoss2,
PacketLoss3, PacketLoss4, PacketLoss5, Avg_PacketLoss
2, 9.8, 9.7, 9.5, 9.6, 9.9, 9.7, 0, 0.2, 0.5, 0.3, 0.1, 0.22
5, 8.4, 8.1, 7.9, 7.8, 8.0, 8.04, 1.2, 1.5, 2.1, 2.3, 1.8, 1.78
```

UDP throughput (Mbps) and packet loss (%) are logged separately.

6.4 Analyzing Data with Python

To automate analysis and generate plots, we will use a Python script.

6.4.1 Create iperf_plot.py yourself (Each time you run the script at the same distance, it appends a new row. For plotting, use the last row only (most recent run).)

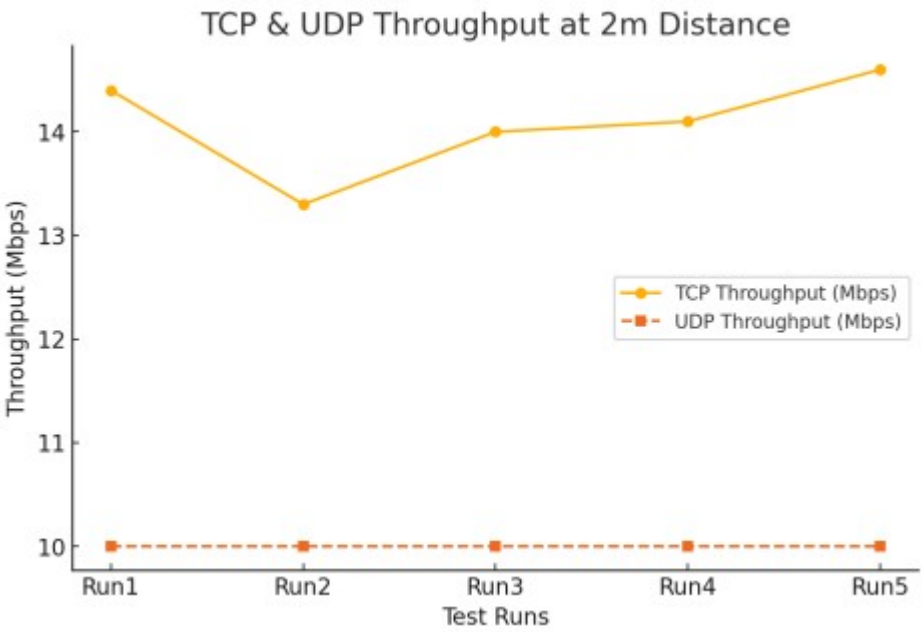
The VM does not have pip yet. Install it and the plotting libraries once:

```
sudo apt install -y python3-pip
pip3 install --break-system-packages pandas plotly "kaleido==0.2.1"   # kaleido
0.2.1 writes PNGs without a Chrome download
```

On the VM, in the same folder as the CSV files, create a new script `iperf_plot.py` with pandas + plotly (as in Part 1). It should read every `iperf_tcp_<d>m.csv` / `iperf_udp_<d>m.csv` pair, take the last row of each, and for each distance draw one figure with Run 1–5 on the x-axis and throughput (Mbps) on the y-axis, TCP and UDP as two lines with a legend, saved as a PNG (e.g. `iperf_2m.png`). Five distances → five figures. This script is one of your Part 2 deliverables.

TCP & UDP Throughput vs Distance

- TCP throughput should gradually decrease with distance.
- UDP throughput may drop suddenly with packet loss.



Example plot for Fixed Laptop VM near router, ESP32-S3 at 2metres from laptop (Distance bw Router and ESP32-S3 is 2m)

Observations from above plot

- TCP throughput fluctuates slightly between 13.3 Mbps and 14.6 Mbps, with an average of 14.08 Mbps.
- UDP throughput remains stable at 10 Mbps, showing no packet loss.

This shows that at close range (2m), the network is stable, with minimal TCP variations and no UDP packet loss.

More Questions from Part 2

After analyzing the data, students should answer the following:

1. How does distance affect TCP and UDP throughput?
2. At what distance does significant packet loss occur for UDP?
3. Why does UDP experience more packet loss than TCP?
4. What happens if we increase the UDP bandwidth (`-b 100M`)?
5. Would performance be different on 5 GHz Wi-Fi vs. 2.4 GHz?

Total of 14 Questions(9 in Part 1 in script main.py, 5 in Part 2)

8. Demo and Code Grading Rubric

Points	Description
10 (Demo)	Part 1: main.py runs and produces the bar chart with error bars; 10 samples per location, 1 s apart, at 5 locations of which at least 3 differ clearly; mean and standard deviation computed correctly.
15 (Demo)	Part 2: iPerf 2 server on the VM and the ESP32-S3 client working (a TCP and a UDP test shown live); iperf_logger.sh run at 5 distances with the resulting CSV files; iperf_plot.py produces the five TCP/UDP throughput plots.
5 (Submission)	Complete code, plots and CSV files, answers to all 14 questions, and an explicit

	acknowledgment of all assistance received (AI tools, people, websites); no exposed credentials.
70 (Interview)	Answer questions posed by the TA/CP about this lab to their satisfaction, demonstrating that you really understood each part, and did not just do them mechanically.
100	**Total**

Submission: each student submits via Brightspace one zip containing `main.py`, `signal_strength.png`, `iperf_plot.py`, the five throughput PNGs, the `iperf_tcp_<d>m.csv` and `iperf_udp_<d>m.csv` files, and `README.md` with the answers to the 9 Part 1 questions (from main.py) and the 5 Part 2 questions, plus an acknowledgment of assistance. The ESP32-S3 firmware is not submitted. Before submitting, remove Wi-Fi passwords from `credentials.h` and from any screenshots.